

Paper Writeup(s) to be Submitted before 22 Jan

Aaryan Bhagat

Student ID: 862468325

Idle Port Scanning and Non Interference Analysis of Network Protocol Stacks Using Model Checking

This 2010 paper is on idle scanning. It first of all creates their own model and to bypass it brings 2 novel ways of idle scanning in which you can get quite a detail of the victim behind a subnetwork without sending packet inside that network. This also they have extended to ports, where you can check if a port is live despite it being behind a firewall. The authors then provide a model checking way which can mitigate these attacks.

Pros: I am impressed by the problem the authors went to tackle, there has been work done in detection of network scans but modelling and mitigating idle scans was not done before. Moreover after explaining their model checking to mitigate idle scanning, the authors even gave counterexamples where idle scanning was being performed. We also see a limited implementation performed by the researchers in SAL and also setting up a test network for the 2 attack vectors based upon RST rate limits and SYN cache.

Assumptions: The paper is very clear about its assumptions because of the model it has devised. Reply to SYN/ACK packet is drop for open ports and RST for closed ports. Excluding ICMP and UDP from split SYN cached version of the model. The cache buffers for the split SYN models have single entry each.

Problems/Cons: Although the paper is quite complete where a model checking is presented and its limitations discussed I think the authors have not varied in packet loss factor into their analysis and how that can change the RST rate limiting example for instance. The paper also has not gone deeper to explain other modes of rate limiting, specifically the adaptive ones, it has for now only confined itself to fixed rate limiting but also not explaining where the fixed numbers come from. Regarding the SYN cache buffers, I have an issue that the authors have not discussed the various implementations in detail and have mainly relied on the FreeBSD one.

TCP Stealth: Secure Server Authorization

This was not originally a research paper but an internet draft, sort of a manifesto (actually called RFC) as to a new standard of TCP should be normalised, a new standard which obscure themselves from obscure TCP clients. It does this by adding a new variable called authorisation token in the TCP handshake process.

Working: It sends a unique authorisation token which are then XOR'd in various manner to get a final 32-bit ISN, this process is then repeated. There is another variant in the paper which also tries to protect integrity of the payload but the essentials is just the same. Duplicate ISN segments will follow the TCP TIME-WAIT state.

Assumptions: The biggest assumption in this paper is that both the client and the server have agreed on a shared secret. This is essential for the authorisation token to work.

Problems/Cons: It cannot protect to passive port scans, where the attacker is acting just as a listener and see which client is connecting to which server. Authorisation tokens require a shared secret, so a shared secret needs to be strong. If the middleware in the whole network path modify the TCP packet, it can create problems, so one needs to double check on that too.

My biggest problem with this paper is not its implementation but the mindset, as someone who really does like adhering to the network stack, I do insist on adding any sort of cryptographic checks above the TCP stack and not inside it. However, even if you keep it aside for the moment, in modern days, many single server domains are public, the services they run are on default ports, if that is not the case, the attacker can just listen in a network and find a machine A which is acting as a client to our main victim server and the attacker can just try to infiltrate the machine to finally know that to which port of the victim server is the machine A connected. So even if one does apply TCP stealth there is not a lot of overhead at present for the attacker to cross and identify the active ports.